Ders 20 Genişletilmiş Notu: Böl ve Yönet Yaklaşımı, Merge Sort ve Quick Sort'a Giriş

Ders 20 Genişletilmiş Notu: Böl ve Yönet Yaklaşımı, Merge Sort ve Quick Sort'a Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Algoritma Tasarlarken Basit Düşünmek
- 2. Böl ve Yönet Fikri
- 3. İkili Arama: Benzer Ama Aynı Değil
- 4. Böl ve Yönet Algoritmalarının Üç Aşaması
- 5. Özyineleme Neden Doğal Çözüm, Ne Zaman Gereksiz?
- 6. Özyineleme Hatırlatması: Faktöriyel
- 7. Ödev Sorusu: İki Grafın Birleştirilmesi
- 8. Elle Örnek Çözme Alışkanlığı
- 9. İki Sıralı Dizinin Birleştirilmesi (Merge)
- 10. Merge Sort Mantığı
- 11. mergeSort Fonksiyonu ve Adım Adım İzleme
- 12. Neden Yedek Dizi Gerekiyor?
- 13. merge Fonksiyonu
- 14. Merge Sort Karmaşıklığı
- 15. Merge Sort'un Özellikleri
- 16. Quick Sort'a Giriş

1. Algoritma Tasarlarken Basit Düşünmek

Hoca konuya girmeden önce algoritma tasarımıyla ilgili genel bir tavsiye verdi: Basit düşünün. Burada Occam'ın usturasını (Occam's razor) hatırlattı. Aynı sonuca ulaşan iki çözümden basit olanı tercih edilmelidir. Hocaya göre öğrenciler sık sık "bu kadar basit olacağını düşünmedik, altında mutlaka bir şey vardır dedik" diyor. Oysa pek çok durumda problem çok basit bir şekilde çözülebiliyor. Ne kadar basit düşünürseniz problemleri o kadar rahat çözersiniz.

2. Böl ve Yönet Fikri

Bu derste algoritma tasarımında çok kullanılan, karmaşıklığı azaltan ve verimliliği artıran bir yaklaşım anlatıldı: böl ve yönet (divide and conquer).

2.1 Roma'dan Gelen Benzetme

Hoca yaklaşımın adını Roma devrindeki bir yöntemle açıkladı. Kalabalık bir ordu üzerinize geliyorsa, bu orduyu bölerseniz ya da birbirine düşürürseniz yönetmek kolaylaşır. İnsanlar bütün ve kalabalık olduklarında onlarla uğraşmak zordur. Bölündüklerinde idare etmek kolaylaşır. Soru şudur: Bu fikri algoritma tasarımında nasıl kullanırız?

2.2 Klasik Sıralama Algoritmaları Neden $O(n^2)$?

Bugüne kadar görülen sıralama algoritmaları, heap sort hariç, en kötü durumda (worst case) $O(n^2)$ karmaşıklığındaydı. Hocaya göre bunun sebebi, sıralama yapılırken dizinin bütünüyle ilgilenilmesidir.

Seçmeli sıralamayı (selection sort) düşünelim. Dizinin tamamına bakar, en küçük elemanı bulur ve ilk elemanla yer değiştirirsiniz. Sonra dizinin kalanının tamamına bakar, yine en küçüğü bulursunuz. Bu böyle devam eder.

Eklemeli sıralamada (insertion sort) ise her eleman kendinden öncekilerle karşılaştırılarak yerine konur. Dizi çok karışık değilse selection sort'tan biraz daha başarılıdır. Kabarcık sıralamasıyla (bubble sort) birlikte bu üç yöntemin ayrıntıları farklıdır. Ama genel mantık aynıdır: Her adımda bütün bilgi üzerinde iş yapılır. Hoca, selection ve insertion'da her seferinde tam olarak dizinin tamamına bakılmadığını da ekledi. Önemli olan, genelde hep bir bütün üzerinde çalışılmasıdır.

2.3 İlk Örnek: Küçükler Bir Tarafa, Büyükler Bir Tarafa

Hoca şu soruyu sordu: Dizinin tamamına bakmak yerine şöyle yapsak ne olur? Dizinin ilk elemanı 4 olsun. 4'ten küçükleri bir tarafa, büyükleri bir tarafa yerleştirelim. Sonuç şudur:

```
2 1 3 | 4 | 7 15 11 5
```

Bu noktada iki şey kazanılmıştır:

- 4 yerine yerleşmiştir. Dizi sıralandığında 4 burada duracaktır.
- Solda 4'ten küçükler, sağda 4'ten büyük veya eşitler vardır.

Şimdi `2 1 3` parçasını kendi içinde sıralamak için sağ tarafa bakmaya gerek var mı? Yok. `7 15 11 5` parçasını sıralarken sol tarafa bakmaya gerek var mı? Yine yok. Dizi bölünmüş, bölündükten sonra her parçanın içinde ayrı ayrı işlem yapılmaya başlanmıştır.

Aynı iş parçaların içinde de yapılabilir. `2 1 3` için 2'den küçükler bir tarafa, büyükler bir tarafa alınırsa 2 yerine yerleşir. 1'in yerini bulmak için 2'nin sonrasına bakmak gerekmez. `7 15 11 5` için 7'den küçükler bir tarafa, büyükler bir tarafa alınırsa 7 de yerine yerleşir. Artık 15 ve 11'i sıralamak için 7'nin öncesine bakmak gerekmez. Çünkü 7'den küçüklerin solda kaldığı bilinir, 15 ve 11 ise 7'den büyüktür.

2.4 Bir Milyon Elemanlı Dizi

Hoca kazancın büyüklüğünü şöyle anlattı: Bir milyon elemanınız olsun. Baştaki x sayısı 200 bininci sıraya yerleşsin. Solda x'ten küçük 199.999 sayı, sağda x'ten büyük 800.000 sayı vardır. Bu 199.999 sayıyı sıralarken sağ tarafa hiç bakmazsınız.

Böl ve yönet algoritmalarının temel mantığı budur. Elimdeki bilgiyi bölmek bir fayda sağlıyorsa, böler ve parçalar üzerinde ayrı ayrı çalışırım. Böylece işlem karmaşıklığını da azaltmış olurum.

> **Hocanın vurgusu:** Bölmenin anlamı şudur: Böldükten sonra bir parça üzerinde çalışırken diğer parçayla hiç ilgilenmezsiniz. Sol taraf sağla, sağ taraf solla ilgilenmez.

3. İkili Arama: Benzer Ama Aynı Değil

Hoca, bilinen algoritmalar içinde bu şekilde bölerek iş yapan bir örnek olup olmadığını sordu.

Soru: Shell sort olabilir mi? **Cevap:** Shell sort aslında böyle bir şey değil.

Soru: İkili arama (binary search) olabilir mi? **Cevap:** Hoca bu cevabı onayladı ve bölerek iş yapma mantığının orada da bulunduğunu söyledi. Ama ikili aramanın tam olarak böl ve yönet olmadığını da ekledi.

Dizi sırasız olduğunda bir elemanı bulmak için dizinin tamamına bakılır. Doğrusal aramanın (linear search) karmaşıklığı bu yüzden $O(n)$'dir. İkili arama ise dizinin sıralı olmasından yararlanır. Örneğin aranan $x = 17$ ise ve x ortadaki 9'dan büyükse, sol tarafa hiç bakılmaz. Dizi ikiye bölünür ve bir yarı elenir. "Madem bu dizi sıralı, tek tek bakarak gitmem" denir. Gereksiz yere bir grup bilgiye bakmaktan kurtulunur. Karmaşıklığın $O(\log n)$ olmasının sebebi budur.

Yine de ikili arama böl ve yönet değil, azalt ve yönet (decrease and conquer) yöntemidir. Böldükten sonra elenen yarı üzerinde hiçbir iş yapılmaz, o bölümle iş biter. Hoca örneği, işin tamamına bakmadan da çözüm üretilebildiğini göstermek için verdi. Bu türlerin ayrıntısı önümüzdeki yılki Algoritma Analizi dersinde görülecek.

Veri yapıları dersinde görülen sıralama algoritmaları selection, insertion, bubble, heap ve shell sort'tur. Hedef, verimli sıralama algoritmalarını da görmektir. Bu yüzden böl ve yönet mantığıyla çalışan merge sort ve quick sort anlatılacak. Hoca, diziyi parçalara bölerek çalıştıkları için bu algoritmaların karmaşıklığının $O(n \log n)$ olduğunu söyledi. Quick sort'un en kötü durumu 15. bölümde ayrıca geçiyor.

> **Ders sonrası not:** Hoca bu sırada heap sort'un da aynı şekilde diziyi parçalara bölerek çalıştığını ima etti. Doğrusu, heap sort böl ve yönet algoritması değildir. Yığın (heap) veri yapısını kullanır ve diziyi birbirinden bağımsız çözülen parçalara ayırmaz. $O(n \log n)$ olması bu yüzden farklı bir sebepten gelir.

4. Böl ve Yönet Algoritmalarının Üç Aşaması

Böl ve yönet algoritmaları üç aşamadan oluşur:

- **Böl (divide):** Problem daha küçük parçalara bölünür. Bölme şekli algoritmaya göre değişir. Her algoritmada ihtiyaca göre farklı bir bölümleme olabilir.
- **Yönet (conquer):** Parçalar birbirine karıştırılmadan, her biri kendi içinde çözülür.
- **Birleştir (combine):** Parçaların çözümleri birleştirilerek bütün sonuç elde edilir.

Yani n elemanlı bir probleminiz varsa genellikle onu iki parçaya bölersiniz. Tam ortadan bölmek her zaman mümkün değildir. Sonra parçaları kendi içinde çözer, çözümleri birleştirir ve bütün sonucu elde edersiniz.

> **Ek bilgi (derste anlatılmadı):** İşin ağırlığı iki algoritmada farklı aşamadadır. Merge sort'ta bölme yalnız indis hesabıdır, asıl iş birleştirme aşamasındadır. Quick sort'ta asıl iş bölme (bölümleme) aşamasındadır. Pivotun solu ve sağı sıralandıktan sonra birleştirmek için ek iş kalmaz, çünkü parçalar zaten aynı dizide doğru sıradadır.

5. Özyineleme Neden Doğal Çözüm, Ne Zaman Gereksiz?

Hoca 2. bölümdeki örneği tahtada yeniden gösterdi: 4 yerleşti, 2 1 3 ve 7 15 11 5 kaldı. 2 yerleşti, tek sayılar kaldı, 1 ve 3 de yerleşmiş oldu, sol taraf bitti. Sağ tarafta 7 yerleşti: 5 7 15 11. Aynı iş 7'nin solu ve sağı için de tekrar edilir, ta ki dizi tamamen sıralanana kadar. Hoca bu örneğin aslında quick sort olduğunu söyledi. Böl ve yönet mantığını anlamak için quick sort'un güzel olduğunu, bu yüzden ondan başladığını, algoritma olarak önce merge sort'u anlatacağını belirtti.

Sonra sınıfa sordu: Aynı işi tekrar eden bir fonksiyon yazacaksanız bu size neyi hatırlatır?

Soru: Özyineleme (recursion) mi? **Cevap:** Aynen öyle.

Böl ve yönette önce bütün ikiye bölünür, sonra bölünen ikiye bölünür, bu böyle gider. Problem geri gide gide çözülür, sonra toparlanarak geri dönülür. Bu tam bir özyineleme mantığıdır.

> **Hocanın vurgusu:** Böl ve yönet algoritmalarını çözmenin en iyi yolu özyinelemedir. Klasik yaklaşımla, yani döngüyle de çözülebilir, ama çok zahmetli ve gereksizdir.

Hoca bunun karşı tarafını da söyledi. Bazen öğrenciler özyinelemeli problem çözmeye alışıyor ve gerekli gereksiz her problemi özyinelemeyle çözmeye çalışıyor. Özyineleme gerektirmeyen pek çok problem de özyinelemeyle yazılabilir. Ama fonksiyon çağırmanın bir ek yükü (overhead) vardır. Verimli tasarım yapmak, yeri ve zamanı iyi kullanmak istiyorsak bunu hesaba katmalıyız. Özyineleme dediğimizde aynı fonksiyonun defalarca çağrılmasından bahsediyoruz.

> **Hocanın vurgusu:** Gerekiyorsa özyinelemeyi kullanın. Gerekmediğinde bir problemi çözmek için gereksiz yere özyineleme kullanmayın.

6. Özyineleme Hatırlatması: Faktöriyel

Hoca özyinelemeyi hatırlatmak için, kendi sözleriyle "asla özyinelemeli yazılmaması gereken" bir örnek seçti: faktöriyel. Özyinelemeyi anlatmak için çok güzel bir örnektir. Ama döngüyle yazıldığında daha verimli çözülür.

$5! = 5 \cdot 4!$ ve bu da $5 \cdot 4 \cdot 3!$'e eşittir. Yani 4 faktöriyeli çözmek için 3 faktöriyel, 3 faktöriyeli çözmek için 2 faktöriyel, 2 faktöriyeli çözmek için 1 faktöriyel kullanılır. 1 faktöriyel de 1 olduğu için problem çözülmüş olur. Genel bağıntı şudur:

$$n! = n \cdot (n-1)!, \qquad 1! = 1$$

Fonksiyonun kendi kendini çağırması sonsuza kadar sürmez, bir durma noktası vardır. Özyinelemeli bir fonksiyonda iki parça bulunmalıdır:

- **Durma noktası (base case):** Fonksiyon buraya gelince durur, daha fazla çağrı yapmaz. Faktöriyelde `n == 1` ise 1 döndürülür.
- **Özyinelemeli kısım (recursive case):** Fonksiyon kendini daha küçük değerle çağırır. Faktöriyelde `n * faktoriyel(n - 1)` döndürülür.

Hoca $n < 1$ değerlerinin aslında verilmemesi gerektiğini, bunların ana programda durdurulması gerektiğini söyledi. Aşağıdaki kodda `main` tamamlanmıştır. Kod `gcc -Wall` ile uyarısız derlendi ve çalıştırıldığında 6 yazdı.

```c
#include <stdio.h>

int faktoriyel(int n)
{
    if (n == 1)
        return 1;                    /* durma noktası (base case) */
    else
        return n * faktoriyel(n - 1); /* özyinelemeli kısım (recursive case) */
}

int main(void)
{
    int n = 3;

    if (n < 1)                       /* n < 1 ana programda engellenir */
        return 1;
    printf("%d\n", faktoriyel(n));
    return 0;
}
```

> **Hocanın vurgusu:** Durma noktasını çok düzgün belirlemelisiniz. Durma noktası bütün durumları karşılamalıdır. Bazı durumlarda ucu açık kalırsa fonksiyon kendini çağıra çağıra gider.

Bunun sonucu C programlarında görülen yığın taşması (stack overflow) hatasıdır. Fonksiyon çağrıları yığın (stack) bölgesini kullanır. Durma iyi ayarlanmadıysa fonksiyon kendini çağırmaya devam eder, bir süre sonra bu bölge dolar ve hata alınır.

> **Ek bilgi (derste anlatılmadı):** Hocanın $n < 1$ uyarısının sebebi de budur. Bu koddaki durma koşulu yalnız `n == 1` olduğu için $n = 0$ verilirse $n - 1$ değeri 1'e hiç ulaşmaz. Fonksiyon kendini azalan değerlerle çağırmayı sürdürür. Bu, ucu açık kalan bir durma noktasına örnektir.

6.1 $F(3)$ Çağrısının İzlenmesi

Hoca faktöriyel fonksiyonuna kısaca F dedi ve $n = 3$ için çalışmayı adım adım gösterdi:

- $F(3)$ çağrılır. n 1 değildir, $3 \cdot F(2)$ hesaplanmak istenir.
- $F(2)$ çağrılır. n 1 değildir, $2 \cdot F(1)$ hesaplanmak istenir. O anki ifade $3 \cdot 2 \cdot F(1)$'dir.

- $F(1)$ çağrılır. $n = 1$ olduğu için 1 döner. Çağrılar burada durur.
- Geri dönüş başlar. $F(2) = 2 \cdot 1 = 2$ olur.
- $F(3) = 3 \cdot 2 = 6$ olur. Ana programa 6 döner.

Yani 3, 2'yi çağırır. 2, 1'i çağırır. 1'in değeri yerine konur ve toparlaya toparlaya geri dönülür. Böl ve yönet algoritmalarında esas hedef de budur: Parçalara bölüp her parça için aynı işlemi tekrar eden bir tasarım sağlamak.

7. Ödev Sorusu: İki Grafın Birleştirilmesi

Sınıftaki soru üzerine hoca bir önceki ödevdeki graf birleştirme sorusunun mantığını anlattı. Bu konu dersin merge başlığıyla da bağlantılıdır, çünkü hoca birazdan bunu da bir merge işlemi olarak anacaktı.

Elimizde iki graf var ve iki grafta ortak düğümler bulunuyor. Hoca önce sayılarla bir çizim yaptı: Birinci grafta 0, 1 ve 2'ye bağlı, 1 de 3'e bağlı. İkinci grafla ortak olan düğüm 1 ise, 1'in ikinci graftaki komşusu olan 4 birleşik grafa eklenir, 4'ün bağlı olduğu 6 da onunla birlikte gelir. İş bittiğinde elde tek bir graf olmalıdır. Hocaya göre "merge"den kastedilen buydu.

Hoca işlemi harfli örnekle şöyle özetledi: Bir tarafta B var, öbür tarafta da B varsa, B'nin bir tarafta bağlı olduğu ama diğer tarafta olmayan düğümlere bakılır. Örneğin B, C ve E'ye bağlı olsun. Diğer grafta C ve E zaten varsa yapılacak bir şey yoktur. B'nin bağlı olduğu F diğer tarafta yoksa F alınır ve oradaki B'ye bağlanır.

> **Ders sonrası not:** Hoca bu işlem için "kesişim işlemi" dedi. Doğrusu, anlatılan işlem kesişim değil birleşimdir. Ortak düğümler üzerinden, yalnız bir tarafta bulunan düğüm ve kenarlar da sonuca eklenir.

Şekille ilgili sorular da soruldu:

Soru: Çizdiğiniz şekil ağaç gibi görünüyor. **Cevap:** Ağaç değil, graftır. Hoca örneği önce gerçek bir graf olarak büyük çizdiğini, çok karışık olunca basit tuttuğunu söyledi.

Soru: Sol grafta B ve C bağlı olsaydı, sağda da bağlı olması gerekir miydi? **Cevap:** Hoca önce evet dedi. Asıl vurgusu şuydu: Graf size giriş bilgisi olarak dışarıdan veriliyor, örneğin bir matris olarak. Tasarım yaparken grafın ne kadar karmaşık olduğunun bir önemi yok. B ile C'nin bağını kurmanız istenmiyor, bu bağ size zaten öyle gelir. Bakmanız gereken, bir tarafta olup öbür tarafta olmayan düğümlerdir.

Hoca aradan sonra bu cevaba döndü. Önce B ile C'nin bağlı olamayacağını söyledi, sonra hemen düzeltti: B ve C'nin bağlı olmasında bir sıkıntı yoktur. Bu bağ yeni grafta da verilmiştir ve siz ikisini birleştirirsiniz.

Bir öğrenci uygulamadaki bir zorluktan da söz etti: Düğümler kopyalandığı için bir düğüm sorgulandığında, örneğin 3 sorgulandığında, hem dizideki 3'ün hem de diğerlerinin gösterdiği düğümlerdeki 3'ün sayacını tek tek bulup artırmak gerekiyordu.

8. Elle Örnek Çözme Alışkanlığı

Ödev konuşmasından çıkan akademik tavsiye şudur: Problemin girişini ve çıkışını kendiniz belirleyebilmelisiniz. Girişte ne almalıyım, çıkışta ne vermeliyim sorusunu öğrencinin kendisi çözmelidir. Hoca "gerçek dünyada kim size örnek verecek?" diye sordu. Bir örnek verilmedi diye takılıp kalmamak gerekir.

> **Hocanın vurgusu:** Karmaşık düşünmeyin. Bir problemi anlamak için mutlaka elle örnek çözün.

Hoca kendi öğrenciliğinde çok faydasını gördüğü yöntemi şöyle anlattı:

- Önce çok basit, problemsiz 2-3 örneği elle çözün. Orada ne olduğunu anlamaya çalışın.
- Sonra 2 karmaşık örneği elle çözün. Az önce konuşulan "uçlarda bağlı olursa ne olur" meselesi böyle bir örnektir.

Böylece problemin kapsamını genel olarak anlamış olursunuz. Bütün bunlara rağmen kafanızda bir şey oturmadıysa, o zaman tabii ki sorun.

9. İki Sıralı Dizinin Birleştirilmesi (Merge)

Aradan sonra hoca, merge sort'a geçmeden önce birleştirme (merge) algoritmasını hatırlattı.

Elimizde kendi içinde sıralı iki dizi var ve bunları sıralı tek bir dizide birleştirmek istiyoruz:

```
A = 2  7  9  11  15     (n_A = 5)
B = 8  17  19           (n_B = 3)
```

Sonuç dizisi 8 elemanlı olacaktır.

> **Hocanın vurgusu:** Dizileri uç uca ekleyip sonra insertion sort veya selection sort çalıştırmak doğru bir mantık değildir. Madem diziler sıralı, doğru olan bunların sıralı olmasından faydalanmaktır.

İlk elemanlar karşılaştırılarak başlanır. Küçük olan sonuca yazılır ve o dizide bir adım ilerlenir:

Adım	Karşılaştırma	Küçük olan	Sonuç dizisi
1	2 ile 8	2	`2`
2	7 ile 8	7	`2 7`
3	9 ile 8	8	`2 7 8`
4	9 ile 17	9	`2 7 8 9`
5	11 ile 17	11	`2 7 8 9 11`
6	15 ile 17	15	`2 7 8 9 11 15`
7	A bitti	kalan 17, 19 taşınır	`2 7 8 9 11 15 17 19`

Küçük olanı yazmak güvenlidir. Hocanın ilk adımdaki gerekçesi şudur: 2, 8'den küçükse 17 ve 19'dan da küçüktür. Bu yüzden 2 yeni dizinin en küçük elemanı olarak yazılabilir. Aynı şekilde 8, 9'dan küçükse 11 ve 15'ten de mutlaka küçüktür.

Hangi dizinin önce bittiğinin eleman sayısıyla ilgisi yoktur. Hoca bunu görmeniz için kısa diziyi özellikle sona kalacak şekilde seçti. Hangi dizi önce biterse, diğerinin kalan elemanları sona eklenir.

Bu algoritma bir sıralama (sort) değildir. Merge, herhangi bir sebeple iki bilgiyi birleştirme işlemidir. Ödevdeki iki grafın birleştirilmesi de bir merge işlemiydi. Burada yapılan ise iki sıralı diziyi birleştirip sıralı tek dizi elde etmektir.

Soru: n_A ve n_B uzunluklu iki dizi için merge işleminin karmaşıklığı nedir? **Cevap:** $O(n_A+n_B)$. Hoca bunu onayladı ve $n_A + n_B$'yi n diye düşünürsek karmaşıklığın $O(n)$ olduğunu söyledi. Karşılaştırır, kalanı taşırsınız, yani n adım iş yapmış olursunuz.

10. Merge Sort Mantığı

Birleştirme sıralı iki dizi için bu kadar avantajlıysa, aynı mantık sırasız tek bir diziyi sıralamak için kullanılabilir mi? Birleştirerek sıralama (merge sort) bu soruya dayanır. Örnek dizi 8 elemanlıdır:

```
7 5 2 4 1 6 3 0
```

Bir şey öğrenmiştik: Sıralama sırasında dizinin tamamına bakmak yerine anlamlı bir şekilde bölmek avantaj sağlıyor. O hâlde dizi iki parçaya bölünsün: `7 5 2 4` ve `1 6 3 0`.

Burada bölmek fiziksel bir işlem değildir. Parçalar yeni bir yere yazılmaz. Yalnız 0'dan 3'e kadar olan bölümü ve 4'ten 7'ye kadar olan bölümü ayrı değerlendireceğim demektir.

Soru: `7 5 2 4` ve `1 6 3 0` birleştirilebilir mi? **Cevap:** Bir öğrenci "Sıralı olsalar ederdik ama bu durumda edemeyiz" dedi. Hoca bu cevabı çok doğru buldu.

Öyleyse bölmeye devam edilir: `7 5` ve `2 4`. Bunlar da sıralı değildir. Peki ne olursa bir parça gerçekten sıralı olur? Tek elemana inildiğinde. Tek elemanlı parça kendi içinde sıralıdır. 7 ile 5 artık birleştirilebilir.

Merge sort'un bütün fikri budur. Önce dizi tek elemanlı parçalara kadar bölünür (split). Sonra bu parçalar birleştirilerek yukarı çıkılır:

```
Bölme:        7 5 2 4 1 6 3 0
              7 5 2 4 | 1 6 3 0
              7 5 | 2 4 | 1 6 | 3 0
              7 | 5 | 2 | 4 | 1 | 6 | 3 | 0

Birleştirme:  5 7 | 2 4 | 1 6 | 0 3
              2 4 5 7 | 0 1 3 6
              0 1 2 3 4 5 6 7
```

Hoca son birleştirmeyi yavaşça gösterdi. `2 4 5 7` ve `0 1 3 6` birleştirilir:

Karşılaştırma	Yazılan
0 ile 2	0
1 ile 2	1
3 ile 2	2
3 ile 4	3
6 ile 4	4
6 ile 5	5
6 ile 7	6
ilk parçada 7 kaldı	7 taşınır

Genel olarak `A[0..n-1]` dizisi `A[0..n/2-1]` ve `A[n/2..n-1]` diye iki parçaya ayrılır ve birer eleman kalana kadar böyle devam edilir. Böl ve yönetin üç aşaması burada şöyledir:

- **Böl (divide):** Dizi birer eleman kalana kadar bölünür.
- **Yönet (conquer):** Bölünen her parça sıralanır.
- **Birleştir (combine):** Sıralanmış parçalar birleştirilir.

Hoca, bilgisayar alanındaki pek çok şeyin çok eski zamanlarda bulunduğunu söyledi ve merge sort'un von Neumann tarafından yapıldığını, hâlâ başarıyla kullanıldığını ekledi.

> **Ders sonrası not:** Hoca tarih olarak 1935 dedi. Doğrusu büyük olasılıkla 1945'tir.

11. mergeSort Fonksiyonu ve Adım Adım İzleme

11.1 Özyinelemeli Kısım

`p` bakılan bölgenin başlangıcını, `r` sonunu gösterir. Ana programdan ilk çağrıda bütün dizi için `p = 0`, `r = n - 1` verilir. Hoca önce özyinelemeli kısmı yazdı: Bölgeyi ikiye böl, merge sort'u iki parça için çağır, sonra iki parçayı birleştir. Sonra durma şartını sordu.

Bir eleman kaldıysa bakacak başka eleman yoktur. Bu yüzden işlem `p < r` olduğu sürece yapılmalıdır. `p == r` demek, bakılan bölgede tek eleman kalmış demektir.

```
void mergesort(int A[], int p, int r)
{
    int q;

    if (p < r)
    {
        q = (p + r) / 2;
        mergesort(A, p, q);
        mergesort(A, q + 1, r);
        merge(A, p, q, r);
    }
}
```

İlk çağrıda $q = (n-1)/2$ olur. Hoca bunun $n/2 - 1$'e karşılık geldiğini söyledi.

> **Ders sonrası not:** Bu eşitlik n çift iken geçerlidir. n tek ise tamsayı bölmesiyle $q = (n-1)/2$ değeri $n/2 - 1$ değil, $\lfloor n/2 \rfloor$ olur. Ayrıca kısa nota göre tahtadaki kodda `q` büyük olasılıkla tanımlanmamıştı (altyazıda okunmuyor). Yukarıda `int q;` eklendi.

11.2 `7 5 2 4 1 6 3 0` Dizisi İçin Çağrı Sırası

Hoca çağrıları tek tek izledi. Aşağıdaki iz, kodla çalıştırılarak doğrulandı:

```
mergesort(0,7)  q=3
  mergesort(0,3)  q=1
    mergesort(0,1)  q=0
      mergesort(0,0)  tek eleman, döner
      mergesort(1,1)  tek eleman, döner
      merge(0,0,1)    -> A: 5 7 2 4 1 6 3 0
    mergesort(2,3)  q=2
      mergesort(2,2)  tek eleman, döner
      mergesort(3,3)  tek eleman, döner
      merge(2,2,3)    -> A: 5 7 2 4 1 6 3 0
    merge(0,1,3)      -> A: 2 4 5 7 1 6 3 0
  mergesort(4,7)  q=5
    mergesort(4,5)  q=4
      mergesort(4,4)  tek eleman, döner
      mergesort(5,5)  tek eleman, döner
      merge(4,4,5)    -> A: 2 4 5 7 1 6 3 0
    mergesort(6,7)  q=6
      mergesort(6,6)  tek eleman, döner
      mergesort(7,7)  tek eleman, döner
      merge(6,6,7)    -> A: 2 4 5 7 1 6 0 3
    merge(4,5,7)      -> A: 2 4 5 7 0 1 3 6
  merge(0,3,7)        -> A: 0 1 2 3 4 5 6 7
```

Hocanın adım adım anlatımında dikkat edilecek noktalar şunlardır:

- `p == r` olan çağrıda `if` sağlanmaz ve fonksiyon hemen döner. Hoca bunu "buradan çıktı, bir aşağı indi" diye anlattı. Sol çağrıdan dönülünce sağ çağrıya, sağ çağrıdan dönülünce `merge`'e düşülür.
- Hoca, $p = 0$, $r = 1$ birleştirmesinden sonra $A[0]$'da 5, $A[1]$'de 7 olduğunu anlattı.
- Hoca, $p = 2$, $r = 3$ için q'nun 2 olduğunu söyledi.
- Hoca anlatım sırasında `merge(0,1,3)` adımını atladı. `merge(4,4,5)`'ten sonra bunu fark edip geri döndü ve düzeltti: Sol yarı `2 4 5 7` olmalıdır. Yukarıdaki iz doğru sırayı gösterir.

  Ders sonrası not: Altyazıda $A[1]$ için "1 var" geçiyor. Doğrusu 7'dir. q hesabı için de "3 bölü 2, 2" geçiyor. Doğrusu $(2 + 3)/2 = 2$'dir.

Hoca izlemeyi toparlarken şunu söyledi: Her seferinde önce sol parça sonuna kadar bitirilir. 0-7 için 0-3, onun için 0-1, onun için 0-0 çağrılır. Sol bitince sağa geçilir. İki taraf da bitince birleştirilir. En sonunda 0-3 ve 4-7 parçaları birleştirilerek iş tamamlanır.

> **Hocanın vurgusu:** Bu çağrı sırasını anlamak önemlidir. Özyinelemeli algoritma konusunda sıkıntı yaşamamanız gerekir.

12. Neden Yedek Dizi Gerekiyor?

Hoca burada merge sort'un bir "hilesi" olduğunu söyledi: Merge sort yedek dizi kullanır. Sıralama algoritmalarında genelde "yedek dizi kullanmayın" denir. Selection, bubble, insertion, quick ve heap sort hep aynı dizi üzerinde işlem yapar. Yalnız merge sort, birleştirme aşamasında yedek dizi kullanır.

Sebebi şu aşamada görülür. `2 4 5 7` ve `0 1 3 6` ayrı iki dizi değildir, aynı A dizisindeki iki bölgedir:

```
A: 2 4 5 7 0 1 3 6
```

0 ile 2 karşılaştırılır. 0 küçüktür ve en başta olması gerekir. Ama 0 oraya alınıp 2 de 0'ın yerine konursa ikinci bölgenin sırası bozulur. Tasarım karmaşıklaşır ve ek bir yük gelir. Bu yüzden birleştirme yedek bir dizide yapılır.

> **Ders sonrası not:** Hoca aynı dizi üzerinde birleştirmenin "mümkün olmadığını" söyledi, hemen ardından tasarımın karmaşıklaştığını ekledi. Doğrusu ikincisidir: Aynı di-

zide birleştirme imkânsız değildir. Örneğin elemanlar kaydırılarak yapılabilir, ama bu ek iş getirir. Basit tasarımın yedek dizi kullanmasının sebebi budur.

13. merge Fonksiyonu

`merge(A, p, q, r)` şu anlama gelir: `A[p..q]` ve `A[q+1..r]` sıralı parçalarını birleştir.

Yedek dizinin boyutu. Eleman sayısı $n = r - p + 1$'dir. Hoca bunu sınıfa sordu ve örnek verdi: $p = 3$, $r = 5$ ise 3, 4 ve 5 dahil olduğu için 3 eleman vardır. Yedek `B` dizisi için bu kadar yer ayrılır.

İki parça üzerinde ilerleme. İki dizi parçası da aynı A dizisindedir. `i` birinci parçada `p`'den `q`'ya, `j` ikinci parçada `q + 1`'den `r`'ye ilerler. Küçük olan eleman `B`'ye yazılır ve o parçada ilerlenir.

Bitmeyen parçanın taşınması. Döngüden ya birinci ya ikinci parça bittiği için çıkılır. Bitmeyen parçanın kalanı `B`'ye taşınmalıdır. Hoca bunu `if-else` yerine iki ayrı `while` ile yazdı. `if-else` ile de olurdu, iki yol da aynı kapıya çıkar. İki `while` yalnızca bir kontrol fazla yapar ama daha sadedir.

Geri kopyalama. Sıralamanın A üzerinde olması istenir. Hocanın örneğiyle, A'da `5 7` ve `2 4` varsa `B`'de sıralı `2 4 5 7` oluşmuştur. Bu bilgi `A[p..r]` bölgesine geri kopyalanmalıdır. Bunun için `i` `p`'den, `k` 0'dan başlatılır, `i` `r`'ye kadar gider ve `A[i] = B[k]` yapılır. İş bitince `B` serbest bırakılır (`free`).

Aşağıda `merge` ve `mergesort` tam program olarak verilmiştir. `main` tamamlanmıştır. Kod `gcc -Wall` ile uyarısız derlendi ve çalıştırıldığında `0 1 2 3 4 5 6 7` yazdı.

```c
#include <stdio.h>
#include <stdlib.h>

void merge(int A[], int p, int q, int r)
{
    int n, i, j, k;
    int *B;

    n = r - p + 1;
    B = (int *) malloc(n * sizeof(int));
    if (B == NULL)
        exit(1);

    i = p; j = q + 1; k = 0;
    while ((i <= q) && (j <= r))
    {
        if (A[i] <= A[j])
            B[k++] = A[i++];
        else
            B[k++] = A[j++];
    }
    while (i <= q) B[k++] = A[i++];
    while (j <= r) B[k++] = A[j++];

    for (i = p, k = 0; i <= r; i++, k++)
        A[i] = B[k];
    free(B);
}

void mergesort(int A[], int p, int r)
{
    int q;

    if (p < r)
```

```
    {
        q = (p + r) / 2;
        mergesort(A, p, q);
        mergesort(A, q + 1, r);
        merge(A, p, q, r);
    }
}

int main(void)
{
    int A[] = {7, 5, 2, 4, 1, 6, 3, 0};
    int n = sizeof(A) / sizeof(A[0]);
    int i;

    mergesort(A, 0, n - 1);
    for (i = 0; i < n; i++)
        printf("%d ", A[i]);
    printf("\n");
    return 0;
}
```

Ders sonrası not: Tahtadaki başlangıç ataması büyük olasılıkla `i = k = p;` biçimindeydi (kısa nota göre; altyazıda bu satır okunmuyor). Bu biçimde `k` değeri `p > 0` olduğunda `B` dizisinin dışına yazar. Bu sürüm derlenip bellek denetimiyle (AddressSanitizer) çalıştırıldığında `heap-buffer-overflow` hatası verdi. `B` 0'dan başladığı için doğrusu `k = 0;`'dır.

Ders sonrası not: Hoca bitmeyen parçayı taşıma koşulunu anlatırken "i, q'dan küçükse" ve "j, r'den küçükse" dedi. Doğrusu "küçük eşit"tir (`i <= q`, `j <= r`). Parçanın son elemanı da taşınmalıdır. `<` ile yazılan sürüm ders dizisinde yanlış sonuç üretti.

14. Merge Sort Karmaşıklığı

Hoca algoritmaların analizinin asıl olarak Algoritma Analizi dersinin konusu olduğunu, buna çok girmeyeceğini söyledi. Ama merge sort'u bu kadar övdükten sonra karmaşıklığının nasıl hesaplandığını göstermek istedi. Anlaşılması kolay olsun diye n ikinin kuvveti kabul edildi.

14.1 Yineleme Bağıntısı

Durma durumu. $n = 1$ ise yapılan iş sabittir. Yalnız bir karşılaştırma yapılır ve n'e bağlı bir iş yoktur. Buna c sabiti denir.

Özyinelemeli durum. $n > 1$ ise iki iş yapılır. Merge sort $n/2$ büyüklüğünde iki parça için iki kez çağrılır, bir kez de merge yapılır.

Soru: Merge işleminin karmaşıklığı neydi? **Cevap:** n.

Hoca, sabit önemli olmadığı için bunun $c \cdot n$ de yazılabileceğini söyledi. Yineleme bağıntısı (recurrence relation) şudur:

$$T(n) = \begin{cases} c & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$

Üstteki satır durma durumunu (base case), alttaki satır özyinelemeli durumu (recursive case) gösterir.

14.2 Geriye Doğru Yerine Koyma

$T(n/2)$ bilinmediği için o da çözülmelidir. Geri giderek yerine koyma yöntemi (backward substitution) kullanıldı:

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &= 2\left[2T(n/4) + n/2\right] + n = 4T(n/4) + 2n \\ &= 4\left[2T(n/8) + n/4\right] + 2n = 8T(n/8) + 3n \end{aligned}$$

Buradan genel hâl yazılabilir:

$$T(n) = 2^i\, T\Big(\frac{n}{2^i}\Big) + i \cdot n$$

Durma $T(1)$'de olur, çünkü onun değeri bilinmektedir ve daha gerisi yoktur. $n/2^i = 1$ olması gerekir. Bu da $n = 2^i$, yani $i = \log_2 n$ demektir. Yerine konursa ($T(1)$ hoca tarafından 1 alınarak):

$$T(n) = 2^i \cdot 1 + i \cdot 2^i = n + n\log_2 n$$

Büyük O (big O) gösteriminde en yüksek terim alınır:

$$T(n) = O(n \log_2 n)$$

$T(1) = c$ bırakılırsa sonuç $c \cdot n + n\log_2 n$ olur ve mertebe değişmez. Bu kapalı biçim ve genel hâl, $n = 2^0$'dan 2^{20}'ye kadar bağıntı doğrudan hesaplanarak doğrulandı.

Hoca n ikinin kuvveti olmasa da, örneğin 15 ya da 27 olsa da sonucun yaklaşık olarak aynı çıkacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** n ikinin kuvveti değilse parçalar $\lceil n/2 \rceil$ ve $\lfloor n/2 \rfloor$ boyutunda olur. Bağıntı $T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n$ biçimini alır. Çözümün mertebesi yine $n \log_2 n$'dir.

Hoca çözümü özyineleme ağacı gibi de özetledi: $T(n)$ iki tane $T(n/2)$'den, her $T(n/2)$ iki tane $T(n/4)$'ten oluşur ve bu $T(1)$'e kadar iner. Yöntemi genelleştirirsek önce algoritmanın mantığına bakılır. Burada iki kez merge sort, bir kez merge çağrılıyordu. Buradan yineleme bağıntısı yazılır, durma durumuyla birlikte ele alınır ve geri geri gidilerek çözülür. Bu tür bağıntıların diğer çözüm yolları Algoritma Analizi dersinde görülecek.

15. Merge Sort'un Özellikleri

- **Kararlıdır (stable).** Hoca merge sort'un sıralama algoritmaları içinde kararlı olduğunu söyledi.
- **En kötü durum en iyi duruma eşittir.** Merge sort her zaman $O(n \log n)$ karmaşıklıkla çalışır. Quick sort'ta ise dizi sıralıysa en kötü durumda karmaşıklık $O(n^2)$'ye gider. Hoca bunun nasıl olduğunu ileride göstereceğini söyledi. Merge sort'ta böyle bir durum yoktur.
- **Tek dezavantajı yedek dizidir.** Yedek dizinin boyutu bakılan alanın büyüklüğüne bağlıdır. Bir milyon elemanınız varsa her seferinde bir milyonluk yere ihtiyacınız yoktur. Ama sonuçta, son adımda, bir milyonluk yedek diziye ihtiyacınız olur.

  Ek bilgi (derste anlatılmadı): Kararlı (stable) sıralama, eşit anahtarlı elemanların sıralamadan sonra da aralarındaki ilk sırayı korumasıdır. Bu özelliği `merge` içindeki `A[i] <= A[j]` koşulu sağlar. Eşitlikte birinci parçadaki eleman önce yazılır.

  Ders sonrası not: "En kötü durum en iyi duruma eşittir" sözü mertebe içindir. Karşılaştırma sayısı girdiye göre değişebilir, ama iki durumda da mertebe $n \log n$'dir.

16. Quick Sort'a Giriş

Hoca bu derste quick sort'a yalnız giriş yaptı. Ayrıntıların sonraki derste konuşulacağını söyledi.

16.1 Genel Fikir

Hızlı sıralama (quick sort), hocaya göre en çok kullanılan sıralama algoritmasıdır ve böl ve yönetin en etkileyici örneklerinden biridir. Hoca Java kütüphanelerinin de quick sort kullandığını, küçük dizilerde ise eklemeli sıralamaya geçildiğini söyledi. Sebebi şudur: 10-15, hatta 50 eleman için sürekli fonksiyon çağıran özyinelemeli bir algoritma gereksizdir. Insertion sort bu boyutta daha verimlidir. Quick sort'un içinde küçük parçalar sıralanırken de aynı nedenle insertion sort kullanılır.

> **Ders sonrası not:** Java için bu söz büyük olasılıkla yalnız temel tipler (`int`, `double` vb.) için geçerlidir. Nesne dizilerinde büyük olasılıkla merge sort tabanlı bir yöntem (TimSort) kullanılır.

Mantık 2. bölümdeki örnektir: Bir pivot seç. Pivottan küçükleri bir tarafa, büyükleri bir tarafa koy. Böylece bütün diziye bakmak yerine yalnız bir parçaya bakılır. Sonra küçükler ve büyükler kendi içinde değerlendirilir. Pivot olarak genelde dizinin ilk elemanı alınır. Pivot başka seçilirse ne olacağı daha sonra konuşulacak.

16.2 Küçükleri ve Büyükleri Ayırmak: Öneriler

Hoca bir örnek dizi verdi. Dizi "ikinci adımda sıralanmasın" diye özellikle seçilmişti:

```
5 3 1 9 8 2 4 7     pivot = 5
```

Sonra sınıfa nasıl yapacaklarını sordu. Sınıftan iki öneri geldi, üçüncü yolu hoca bunların üzerine kurdu:

- **Pivotu ilerletmek:** Pivot, kendinden küçük bir eleman gördükçe onunla yer değiştirerek ilerler. Örneğin 3 küçükse 3 başa, 5 onun yerine gelir. Hoca bunu yollardan biri olarak kabul etti.
- **Hemen sona göndermek:** 3 ve 1 yerinde kalır. 9 büyüktür ve sona gitmelidir. O hâlde 9 sondaki 7 ile hemen yer değiştirilir, sonra 7 ile ilgilenilir.
- **İki işaretçi:** Biri baştan, biri sondan gelen iki işaretçi (pointer) kullanılır. Yer değiştirme bir küçük ve bir büyük eleman bulununca yapılır.

Pivotu dizinin ortasına koyma fikri de konuşuldu. Hoca pivotun nereye düşeceğinin önceden bilinmediğini, belki üçüncü, belki beşinci gözde olacağını hatırlattı.

Hoca bütün önerilerin doğru olduğunu, üçüncüsünün yalnız biraz daha iyi olduğunu söyledi. Quick sort'ta tercih edilen, yer değiştirmeyi en aza indirmektir.

> **Hocanın vurgusu:** Pivot yokmuş gibi davranın. Yalnız diğer elemanları ayırın, pivotu en sonda bir kez yerine koyun. Pivotu sürekli oynatmayın. Elemanlar gerekmedikçe hareket ettirilmediği için yer değiştirmeden de kazanılır.

16.3 Birinci Bölümleme

`i` soldan ilerler, pivottan küçükleri geçer. `j` sağdan ilerler, pivottan büyükleri geçer. İki işaretçi de durduğunda elemanlar yer değiştirir:

Adım	Olan	Dizi
0	Başlangıç, pivot 5	`5 3 1 9 8 2 4 7`
1	`i`: 3 ve 1 küçük, geçilir. 9'da durur	`5 3 1 9 8 2 4 7`
2	`j`: 7 büyük, geçilir. 4'te durur	`5 3 1 9 8 2 4 7`
3	9 ile 4 yer değiştirir, `i` ve `j` birer ilerler	`5 3 1 4 8 2 9 7`
4	`i`: 8'de durur. `j`: 2'de durur	`5 3 1 4 8 2 9 7`
5	8 ile 2 yer değiştirir, `i` ve `j` birer ilerler	`5 3 1 4 2 8 9 7`
6	`i` ve `j` birbirini geçti, duruldu	`5 3 1 4 2 8 9 7`
7	Pivot son küçük olan 2 ile yer değiştirir	`2 3 1 4 5 8 9 7`

Normalde `i` değeri `j`'den küçük veya eşit olmalıdır. `i`, `j`'yi geçtiyse bakılması gereken bütün elemanlara bakılmıştır. Son küçüğün yerinin nasıl bulunacağı ise koda geçilince görülecek.

Soru: `i`, `j`'yi geçip gittiyse bütün elemanlara bakılmış mıdır? **Cevap:** Evet, sağ taraftakilerin hepsi büyüktür.

Bu noktada iki şey elde edilmiştir. Birincisi, 5 yerine yerleşmiştir. İkincisi, solda 5'ten küçükler, sağda 5'ten büyük veya eşitler vardır. Bu algoritmanın güzelliği de buradadır: `2 3 1 4` kendi içinde, `8 9 7` kendi içinde ele alınır. Diğer tarafla karşılaştırma yapmak gerekmez.

16.4 Özyinelemeli Devam

Pivotun yerine s diyelim. Hoca her adımda `left` ve `right` değerlerini yazarak devam etti. Bu iz çalıştırılarak doğrulandı:

```
(l=0, r=7)  5 3 1 9 8 2 4 7  pivot 5  -> 2 3 1 4 | 5 | 8 9 7    s=4
  (l=0, r=3)  2 3 1 4        pivot 2  -> 1 | 2 | 3 4            s=1
    (l=0, r=0)  tek eleman, işlem yok
    (l=2, r=3)  3 4          pivot 3  -> 3 | 4                  s=2
      (l=2, r=1)  böyle bir aralık olamaz, işlem yok
      (l=3, r=3)  tek eleman, işlem yok
  (l=5, r=7)  8 9 7          pivot 8  -> 7 | 8 | 9              s=6
    (l=5, r=5)  tek eleman, işlem yok
    (l=7, r=7)  tek eleman, işlem yok
Sonuç: 1 2 3 4 5 7 8 9
```

Ara adımlar şöyledir:

- **`2 3 1 4`, pivot 2:** `i` 3'te durur, 3 pivottan büyüktür. `j` 4'ü geçer, 1'de durur. 3 ile 1 yer değiştirir ve `2 1 3 4` olur. `i` artar, `j` azalır, birbirlerini geçerler. 2 son küçük olan 1 ile yer değiştirir: `1 2 3 4`. Pivot 1. gözdedir.
- **`3 4`, pivot 3:** 4, 3'ten büyüktür. Yer değiştirme olmaz ve pivot olduğu yerde kalır. Pivot 2. gözdedir.
- **`8 9 7`, pivot 8:** 9, 8'den büyük, 7 ise 8'den küçüktür. 9 ile 7 yer değiştirir: `8 7 9`. İşaretçiler birbirini geçer. 8 ile 7 yer değiştirir: `7 8 9`. Pivot 6. gözdedir.

Hoca sağ parçaya geçerken diziyi önce yanlış yazdı, sonra kendisi düzeltti. Doğru parça `8 9 7`'dir.

Soru: Ağacın sol tarafının en altında $l = 0$, $r = 3$ yazılmış. $l = 3$, $r = 3$ olması gerekmiyor mu? **Cevap:** Hoca oradaki değerin 3 olduğunu, yazısının kötü okunduğunu belirtti ve düzeltti.

16.5 Bölümlemenin Sonucu

Bölümlemeden sonra dizi üç bölgeden oluşur:

```
A[0] ... A[s-1]      A[s]      A[s+1] ... A[n-1]
A[s]'den küçük       pivot     A[s]'den büyük veya eşit
```

Soldaki ve sağdaki bölgeler kendi içinde sıralı değildir. Yalnız pivota göre konumları bilinir. Pivot ise sıralı dizideki yerine yerleşmiştir. Eşit elemanların hangi tarafa alınacağı önemli değildir, hoca iki türlü de yapılabileceğini söyledi.

Aynı iş her parça için özyinelemeli tekrarlanır. Bir eleman kaldığında durulur, o elemanın yeri orasıdır. Algoritma tamamen farklı olsa da durma noktası merge sort'takiyle aynı şekildedir.

16.6 QuickSort Sözde Kodu

Hoca kodu kısaca gösterdi ve sonraki derste üzerinde uzun uzun konuşulacağını söyledi. Kod, başlangıç ve son değerleri verilen bir bölge için çağrılır (kısa nottaki tahta biçimiyle):

```
QuickSort(A[l..r])                // ilk çağrı: l = 0, r = n-1
    if (l < r)
        s <- partition(A[l..r])   // pivotun yeni adresi
```

```
        QuickSort(A[l..s-1])
        QuickSort(A[s+1..r])
```

`left`, `right`'tan küçükse yapılacak iş vardır. Eşitse bakılacak tek eleman kalmıştır ve işlem gerekmez. Önce pivotun yeri belirlenir ve dizi, küçükler bir tarafta, büyükler bir tarafta olacak şekilde iki bölüme (partition) ayrılır. Sonra quick sort pivottan küçükler için bir kez, büyükler için bir kez daha çalıştırılır. `partition` fonksiyonunun kodu bu derste verilmedi, bu yüzden burada tam C programı yoktur.

Dersin Toparlaması

Bu derste böl ve yönet yaklaşımının temel fikri görüldü: Problemi anlamlı parçalara bölmek, bir parça üzerinde çalışırken diğerine hiç bakmamak ve sonuçları birleştirmek. Aynı iş her parçada tekrarlandığı için özyineleme bu yaklaşımın doğal aracıdır. Ama gerekmeyen yerde fonksiyon çağrısının ek yükü göze alınmamalıdır. İkili arama da bölerek çalışır, ama elenen yarıyla bir daha işi olmadığı için azalt ve yönet türündedir.

Merge sort, tek elemanlı parçaların zaten sıralı olması fikrinden yola çıkar. Dizi tek elemana kadar bölünür, sonra $O(n)$'lik birleştirmelerle yukarı çıkılır. $T(n) = 2T(n/2) + n$ bağıntısının geriye doğru yerine koymayla çözümü $O(n \log_2 n)$ verir. Her zaman bu mertebede çalışır ve kararlıdır. Bedeli ise dizi boyu kadar yedek bellektir. Quick sort'ta pivotun bir kez yerine konduğu bölümleme izlendi ve özyinelemeli yapı görüldü. Sonraki derste `partition` kodu, pivot seçimi ve quick sort'un en kötü durumu ayrıntılı ele alınacak. Yineleme bağıntılarının diğer çözüm yolları da Algoritma Analizi dersinde görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.